

Arbeitsblatt 4

Abgabeschluss: 14.01.2021 23:59.59 via GitLab

Abnahme-Labor: 15.01.2021 via Discord¹

- Lest die Dokumente *Rahmenbedingungen* und *FAQ* in der ownCloud.
- Kennzeichnet Fremddcode entsprechend der Vorgaben.
- Achtet auf korrekte Quellenangaben für fremde Assets.
- Lösungen durch Fremddcode werden nicht als eigene Leistung gewertet.

Kommt bitte auch ins Abnahme-Labor, falls einige Aufgaben zum Abgabezeitpunkt noch nicht zu 100% funktionieren. Sofern ihr ersthaft probiert habt, die Aufgaben zu lösen, könnt ihr die Laborzeit ggf. noch für Nachbesserungen nutzen.

Vorbemerkungen

Ziel des Arbeitsblattes ist es, dass ihr die Konstruktion komplexer Körper aus geometrischen Grundprimitiven unter Nutzung von Transformationen erlernt. Die beiden Aufgabenstellungen bauen aufeinander auf und sind komplexer als die Aufgaben der vorigen Arbeitsblätter, aber werft nicht gleich die Flinte ins Korn :)

Nutzt beide Wochen zur Bearbeitung des Arbeitsblattes und kommt ins OpenLab, wenn ihr Fragen habt. Versucht nicht zu viel auf einmal, sondern implementiert zunächst Lösungen für kleine Teilaufgaben. Erweitert diese Lösungen dann Schritt für Schritt, aber erst wenn diese auch tatsächlich funktionieren. Copy/Paste von fremden Teil-Lösungen führt in der Regel ins Chaos, versucht stattdessen lieber die fremde Lösung zu verstehen und baut sie dann selbst nach.

Baut Debug-Ausgaben, welche Euch die (tatsächlichen) Werte von Variablen und andere Dinge zeigen, in Euren Code ein. So braucht Ihr bei der Suche nach den Ursachen für vertrackte Fehler nicht blind im Nebel herumstochern, sondern könnt systematisch mögliche Fehlerursachen analysieren.

Und zu guter Letzt: Wenn alles nichts hilft, ist ein Neu-Entwurf manchmal besser als Dauer-Flicken.

Happy Coding!

Aufgabe 1

Erstellt jeweils eine Klasse zur geometrischen Modellierung von Zylindern und eine zur Modellierung von Kreiskegeln.

Das Modell für **Kreiskegel** soll folgende Parameter berücksichtigen:

¹<https://discord.gg/Jr9vNm6W42>

- **radius**: **Radius** der kreisförmigen Grundfläche
- **height**, **Höhe** der Spitze der Mantelfläche über der Grundfläche
- **segments**: Anzahl der Segmente der **Grund- und Mantelfläche**.
- Verwendet den Zeichenmodus **TRIANGLE_FAN** für die beide Flächen

Das Modell für **Zylinder** soll folgende Parameter berücksichtigen:

- **radius**: **Radius** der kreisförmigen Deckflächen oben und unten
- **height**: **Höhe** des Zylinders
- **baseSegments**: Anzahl der Segmente bei der fächerförmigen Zerlegung der **kreisförmigen Deckflächen** in Dreiecke
- **sideSegments**: Anzahl der Segmente bei der ringförmigen Zerlegung der **Mantelfläche** in Dreiecke
- Verwendet die Zeichenmodi **TRIANGLE_FAN** für die Deckflächen und **TRIANGLE_STRIP** für die Mantelflächen

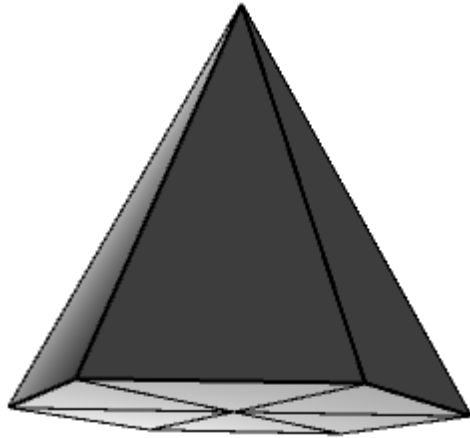
Die Klassen sollen eine Methode **render()** haben, die in der **draw()** Funktion des Sketches aufgerufen werden kann, um einen Zylinder bzw. Kreiskegel mit den entsprechenden Parametern zu zeichnen.

Es soll außerdem möglich sein, alle Parameter der geometrischen Modellierung interaktiv über die Tastatur zu verändern.

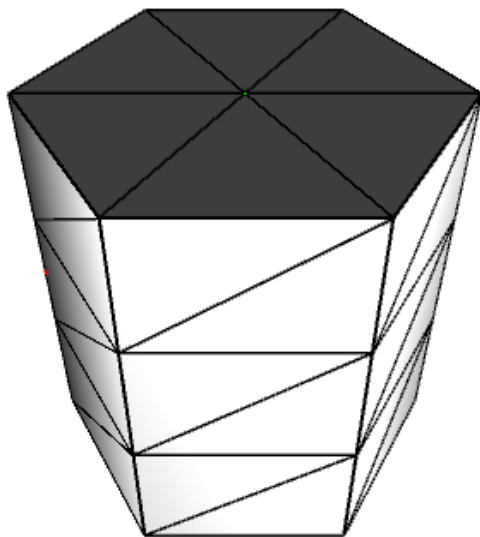
Hinweise

- Es empfiehlt sich zunächst den Kreiskegel zu implementieren.
- Nutzt trigonometrische Funktionen zur Bestimmung der Koordinaten eines Punktes auf einem Kreis.
- Nutzt lokale Koordinatensysteme zur einfacheren Modellierung der Grundkörper.
- Beim Kegel werden die Eckpunkte auf der Kreisfläche auch zur Zerlegung der Mantelfläche in Dreiecke genutzt.

Beispiele



Das Bild zeigt einen Kreiskegel, dessen kreisförmige Grundfläche durch sechs Dreiecke angenähert wurde. Die Mantelfläche besteht ebenfalls aus sechs Dreiecken.



Das Bild zeigt einen Zylinder, dessen obere und untere kreisförmige Deckflächen durch sechs Dreiecke angenähert wurden. Die Mantelfläche des Zylinders wurde in drei ringförmige Segmente untergliedert, die jeweils aus einem Streifen von Dreiecken bestehen.

Aufgabe 2

Erstellt eine Klasse zur Darstellung ein 3D-Koordinatensystem aus Zylindern und Kreiskegeln. Das Zeichnen des Koordinatensystems soll auch hier in einer **render()** Methode erfolgen. Nutzt die Klassen aus Aufgabe 1, um die Primitive, aus denen das Koordinatensystem besteht, zu zeichnen.

Orientiert Euch an der Darstellung von Koordinatensystemen in 3D-Modellierungsprogrammen:

- Ein Pfeil aus einem Zylinder und ein Kreiskegel verdeutlicht die Orientierung einer Hauptachse des Koordinatensystems.
- Alle Achsen des 3D-Koordinatensystems starten im Ursprung des Koordinaten-Systems und zeigen in die positive Richtung.
- Zur Identifikation der einzelnen Achsen werden diese mit den Grundfarben des RGB-Farbraum gekennzeichnet.

